

Sanctuary — Developer Reference

1. Where each change lives, and what to do after editing it

You edited...	Action required	Command
index.html	Reopen with Live Server. No terminal.	None
sanctuary-worker-b2/src/index.js	Redeploy the Worker.	cd sanctuary-worker-b2 then npx wrangler deploy
sanctuary-worker-b2/wrangler.toml	Redeploy the Worker.	npx wrangler deploy
Worker secrets (keys, bucket name, endpoint)	Reset the secret, then redeploy.	npx wrangler secret put SECRET_NAME then npx wrangler deploy
supabase_setup.sql	Re-run in Supabase dashboard. No terminal.	Paste into Supabase SQL Editor → Run
Supabase table/RLS policies	Edit directly in Supabase dashboard, or re-run SQL file.	None, or paste SQL file.

Rule of thumb: anything inside the sanctuary-worker-b2/ folder needs npx wrangler deploy to take effect on the live Worker. Editing the local file alone does nothing until that command runs. Everything else (the app itself, Supabase config) takes effect immediately on save/run with no deploy step.

2. Cloudflare Worker + Backblaze B2 setup — full reference

One-time account setup

No commands — done via dashboards:

1. Sign up at backblaze.com (no card required)

2. Create a private B2 bucket

3. Create an Application Key scoped to that bucket (Backblaze dashboard → App Keys)

→ save Key ID and Application Key immediately, shown once only

4. Note the bucket's S3 endpoint, e.g. s3.us-east-005.backblazeb2.com

→ must be used with https:// prefix in config, even though the

dashboard displays it without one

5. Sign up at dash.cloudflare.com (no card required for Workers)

Project setup (once per machine/clone)

```
cd sanctuary-worker-b2
```

```
npm install
```

Authenticate Wrangler with Cloudflare

```
npx wrangler login
```

Opens browser → click Allow.

Generate the app-to-worker shared secret (once)

```
node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"
```

Copy the output. This is APP_SECRET — used in both the Worker and in index.html (STORAGE_APP_SECRET). Not a Backblaze credential — just a password the app and Worker agree on.

Set Worker secrets (once, or whenever a credential changes)

```
npx wrangler secret put APP_SECRET
```

```
npx wrangler secret put B2_KEY_ID
```

```
npx wrangler secret put B2_APP_KEY
```

```
npx wrangler secret put B2_BUCKET
```

```
npx wrangler secret put B2_ENDPOINT
```

Each prompts for a value, pasted in:

- APP_SECRET → the random string generated above
- B2_KEY_ID → from Backblaze Application Key
- B2_APP_KEY → from Backblaze Application Key
- B2_BUCKET → exact bucket name (e.g. sanctuary-files)
- B2_ENDPOINT → <https://s3.<region>.backblazeb2.com>

Deploy

```
npx wrangler deploy
```

First deploy will prompt to register a workers.dev subdomain — accept, choose a lowercase-letters/numbers/hyphens-only name. Output gives the live Worker URL, e.g.:

```
https://sanctuary-storage.<subdomain>.workers.dev
```

After any Worker code or secret change

```
cd sanctuary-worker-b2
```

```
npx wrangler deploy
```

Local dry-run check before deploying (optional, doesn't go live)

`npx wrangler deploy --dry-run --outdir dist`

App-side config to keep in sync with the Worker

In `index.html`:

`js`

```
const STORAGE_WORKER_URL = 'https://sanctuary-storage.<subdomain>.workers.dev';
```

```
const STORAGE_APP_SECRET = '<same value as APP_SECRET above>';
```

These two must always match what's set on the Worker, or every upload returns 401.

3. Supabase — what's configured and why

Setting	Value / Purpose
Tables	<code>sanctuary_data</code> (all app data, one row per user), profiles (avatar URL)
Row Level Security	Enabled on both tables; policies restrict each user to <code>auth.uid() = id</code>
Auth session storage	<code>sessionStorage</code> , not default <code>localStorage</code> — session clears on full browser close, forcing re-login. Survives page refresh within the same open session.
File storage	Not used. All files (avatars, gallery, music, documents) go through the Cloudflare Worker → Backblaze B2, not Supabase Storage.
Setup script	<code>supabase_setup.sql</code> — paste into Supabase Dashboard → SQL Editor → Run. Idempotent, safe to re-run.

No terminal commands for Supabase — all configuration is SQL pasted into the dashboard, or dashboard UI settings.

4. QA test plan — cross-device / cross-browser sync

Direct answer to your question: **yes, a profile picture uploaded in Edge should appear in Chrome and in the phone app**, because the upload path is Edge → Worker → B2 (cloud), and every device reads from that same cloud URL on login — it is not stored only on the device that uploaded it. The test plan below is how QA verifies this is actually true, not assumed.

Setup

- One test account
- Three environments: Browser A (e.g. Edge, desktop), Browser B (e.g. Chrome, desktop), Browser C (phone, same or different browser)

Test 1 — Profile picture sync

1. Sign in on Browser A. Upload/change profile photo.
2. Confirm toast says success (not "cloud sync failed").
3. Sign in with the same account on Browser B (different browser, same or different device).
4. **Pass condition:** profile photo appears without re-uploading.
5. Repeat on Browser C (phone).

Test 2 — File/document sync (Notebook attachments)

1. On Browser A: create a note, attach a PDF, Word doc, and Excel file.
2. On Browser B: open the same note.
3. **Pass condition:** all three files open inline in the viewer (not download), with correct content.

Test 3 — Music sync

1. On Browser A: upload an audio file.
2. On Browser B: open Music section.
3. **Pass condition:** track appears in library and plays without re-uploading.

Test 4 — Gallery / Vision board / Favourite places photos

1. On Browser A: add a photo to Gallery, Vision board, and a Favourite place.
2. On Browser B: open each section.
3. **Pass condition:** all photos display.

Test 5 — Upload failure visibility

1. Disconnect network mid-upload, or use an invalid Worker URL temporarily.
2. **Pass condition:** toast shows the real failure reason (not silent, not a generic message), and the file/photo still saves locally on that device.

Test 6 — Sign-out behavior

1. Play a track. Click Sign out.
2. **Pass condition:** audio stops immediately, mini-player disappears, no music continues.
3. Close the browser fully (not just the tab) after signing in again.
4. Reopen the app.
5. **Pass condition:** login screen appears — session does not persist across a full browser close.

Test 7 — Streak accuracy

1. Note current streak value.
2. Complete a goal, or write a diary entry, or create a note today.
3. **Pass condition:** streak increases by exactly 1 if yesterday was also active, or resets to 1 if not — never stays frozen, never jumps arbitrarily.

Test 8 — Alarm/reminder ringing

1. Set an alarm with a custom ringtone (uploaded track) for 1 minute ahead.
2. Lock the phone / let the screen attempt to sleep.
3. **Pass condition:** screen stays awake, alarm rings with chosen track, vibrates (Android only — not iOS), visual overlay flashes. If the custom track fails to load, built-in tone plays instead — never silent.

Test 9 — File viewer never downloads

1. Click a PDF, image, Excel file, Word doc, and CSV in Notebook, from at least two different browsers.
2. **Pass condition:** every file type opens in the in-app viewer. No browser download prompt or saved file appears.

Before listing deploy commands, the honest answer to "what's remaining" is more important than the hosting step itself — deploying broken or unfinished work to a real URL doesn't make it launch-ready, it just makes the gaps public. Let me lay out both clearly.

What's actually remaining before this should go live

From everything we've built and fixed, here's the real state:

Area	Status	Risk if skipped
Auth (Supabase)	Done	—
Data sync (Supabase)	Done	—
File storage (B2 + Worker)	Done	—
File viewer (no downloads)	Fixed for PDF/Excel/Word/CSV/images/text	.doc (legacy) and unknown types still show "can't preview" — intentional, not a bug, but QA should confirm that message reads fine
Streak	Fixed, real	—

Area	Status	Risk if skipped
Mobile — Music, Watch, AAY, forms	Fixed	Not yet audited: Gallery lightbox, Projects step rows, Diary toolbar on narrow screens — I flagged this gap explicitly last session and it was never closed
Alarm "can't miss" system	Done (wake lock, vibration, flash, fallback tone)	iOS has no vibration — acceptable, documented
Worker security	Authorization header on upload/delete, open GET (deliberate, filenames are unguessable)	CORS is currently Access-Control-Allow-Origin: * — fine for testing, should be locked to your real domain before public launch
Error visibility	Upload failures show real reasons	—
QA test plan	Written, not yet executed	You haven't told me results of running it

The two things I'd genuinely block launch on:

1. **Mobile audit isn't finished** — I told you this gap existed and we never closed it. "Mobile is the deal here" was your words; shipping with three unaudited sections contradicts that.
2. **CORS wildcard on the Worker** — harmless for testing, but in production it means any website could call your Worker's upload endpoint if they had your APP_SECRET somehow leaked. Tightening it to your real domain is a 5-minute fix, not a redesign.

Neither of those is hosting-related — they're app-readiness items, independent of where you deploy.

Hosting: yes, Vercel is the right call here

Your app is a single static index.html with no build step, no server-side code (the Worker is separate and already deployed on Cloudflare) — that's exactly what Vercel, Netlify, or Cloudflare Pages are built for. Vercel is fine; Cloudflare Pages is arguably more natural since your Worker already lives there and you'd manage one account instead of two. I'll give you both, since it's the same five minutes either way.

Searched the web

Searched the web

Confirmed the commands. Now here's the full picture, no fluff.

Pre-launch blockers (do these before deploying anywhere)

1. **Mobile audit isn't finished.** Gallery lightbox, Projects step rows, Diary toolbar — never tested on narrow viewport.
2. **Worker CORS is wide open.** Access-Control-Allow-Origin: * in sanctuary-worker-b2/src/index.js needs to be locked to your real domain before this is public. Currently anyone with your APP_SECRET could call it from anywhere.
3. **QA test plan exists but hasn't been run.** Run it before hosting, not after — easier to fix on localhost than on a live URL.

Hosting commands — Vercel

`npm install -g vercel`

`cd Sanctuary`

`vercel login`

`vercel`

`vercel` with no flags deploys a **preview**. Test that URL first.

`vercel --prod`

Promotes to production, assigns your real domain.

Custom domain (optional):

`vercel domains add yourdomain.com`

No `vercel.json` needed — a single `index.html` deploys as-is, framework auto-detected as static.

Hosting commands — Cloudflare Pages (alternative, same account as your Worker)

`npx wrangler pages deploy . --project-name=sanctuary`

Run from the Sanctuary folder (the one containing `index.html`). First run creates the project and prompts for confirmation; output gives the live URL. Re-run the same command to redeploy after changes.

Fix CORS before going live (either host)

In `sanctuary-worker-b2/src/index.js`, change:

`js`

`"Access-Control-Allow-Origin": "*",`

to your real deployed domain, e.g.:

`js`

"Access-Control-Allow-Origin": "https://sanctuary.vercel.app",

Then:

```
cd sanctuary-worker-b2
```

```
npx wrangler deploy
```

Order of operations to actually go live

1. Fix CORS in the Worker (above) → `npx wrangler deploy`
2. Run the QA test plan locally (Live Server) — fix anything that fails
3. Deploy the app:
 - `vercel` (preview — test this URL fully first)
 - `vercel --prod` (production, once preview passes)
4. Re-run the QA test plan against the live production URL

Step 4 matters — a Worker pointed at `*` works from localhost, but once CORS is locked to your real domain in step 1, anything still testing from localhost or a preview URL will silently fail uploads. Test against the final production URL last, not first.